FRONT-END

DEVELOPER

2026.03

PORTFOLIO

김예찬 KIM YECHAN

010 - 6383 - 8295
az20058@naver.com
https://web-chan.tistory.com/

PROJECT

01

(주)법틀 - BuptleBiz 2025. 4 ~

기업 법무팀의 빠른 업무 속도 및 안정성을 제공하기 위해 AI를 사용해 계약, 법무 관리 등의 기능을 제공하는 웹 솔루션

02

KORAD - 경화주월 2024. 8 ~ 2024. 10

잘 알려지지 않은 원자력환경공단을 경주 관광지로서 홍보하고, 공단의 관리 안정성 또한 나타내기 위해 제작한 웹앱 프로젝트

03

SKUMARU - 학교 사이트 재구축 2024. 10 ~ 2026. 2

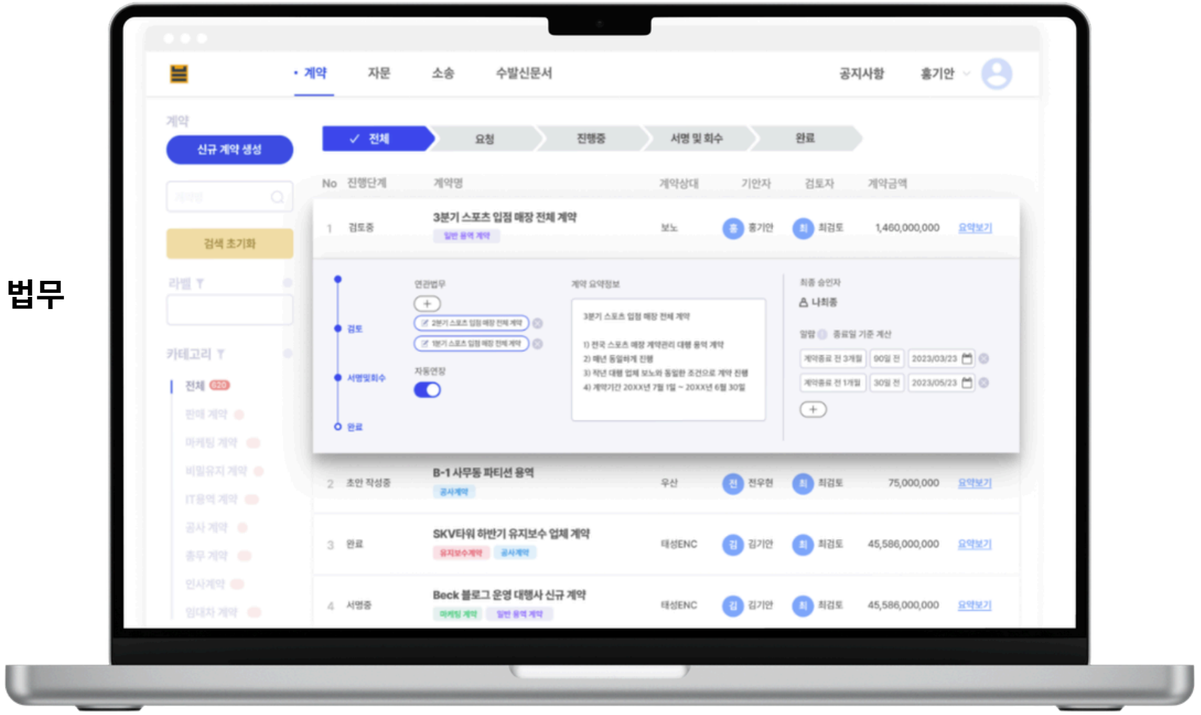
대학교 생활에 있어서 흩어지고 복잡한 정보들을 하나로 모으고, 학우들의 불편함을 개선하기 위해 제작한 프로젝트

PROJECT 01.

(주)법틀 - BuptleBiz

2025. 4 ~ (재직 중)

buptle



프로젝트 개요

기업 법무팀의 **빠른 업무 속도 및 안정성**을 제공하기 위해 **AI**를 사용해 **계약, 법무 관리** 등의 기능을 제공하는 웹 솔루션

React, Next, Typescript, React-Query, Zustand, Tailwind, Django, python, docker, AWS EC2, Recharts, Context-Api, Vitest, MSW

트러블 슈팅 | 1. 배포 지연 개선을 위한 단위 테스트 도입 프로젝트 회고 링크

문제 상황

고객사 요구사항이 증가하면서 통계 페이지 복잡도가 지속적으로 증가했습니다. 이에 따라 코드 수정 시에 사이드 이펙트 발생 확률이 증가하여 안정성은 감소했고, 평균 배포 지연 시간은 1시간 이상이었습니다. 버그 식별 시간을 단축하고, 배포 지연을 단축하기 위해 통계 페이지에 테스트 코드를 도입했습니다.

해결 과정

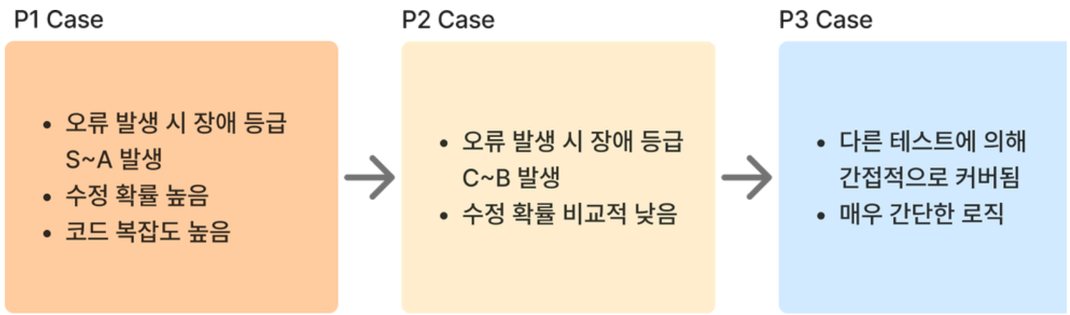
팀의 목표는 '공통 함수 수정에 따른 사이드 이펙트 방지' 였고, 회사 프로젝트와 테스트 코드 작성을 병행해야 했기 때문에 최소한의 리소스로 테스트 코드를 작성해야 했으므로 단위 테스트 적용을 결정했습니다. 테스트 케이스는 우선순위가 높은 순인 P1, P2, P3로 나누어 P1, P2 케이스 위주로 작성했습니다. P1은 오류 발생 시 장애 등급 S를 발생시키며 수정 확률이 높고, P3는 다른 테스트에 의해 간접적으로 커버되며 수정 확률이 낮습니다. AI agent를 활용하여 중요 로직에서 사용되는 함수, 테스트가 중복되는 함수 등을 쉽게 찾아냈고, 각 조건을 중심으로 테스트 우선순위를 분류했습니다. 위와 같은 기준으로 약 200개의 통계 관련 테스트 케이스를 작성했고, 평균 코드 커버리지 82%를 확보했습니다. 이를 통해 배포 지연과 개발 생산성을 개선할 수 있었습니다.

결과 및 배운 점

테스트 코드 도입을 통해 평균 배포 지연 시간은 약 50% 이상 감소했고, 그중 사이드 이펙트에 의한 배포 지연은 거의 0%에 수렴하게 되었습니다. 테스트 코드 작성을 하면 꽤 많은 리소스가 필요하므로, 무조건 높은 커버리지를 확보하는 것보다 팀의 목표와 상황에 따라 적절한 코드 커버리지를 해야 한다는 것을 배웠습니다.

```
-----------------------|---------|----------|---------|---------|----------------
---
File                   | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line
#s
-----------------------|---------|----------|---------|---------|----------------
---
src/pages/statistics   |   85.29 |    82.35 |   88.46 |   85.29 |
 StatisticsPage.tsx    |   91.30 |    85.71 |   100   |   91.30 | 45,78
 StatisticsChart.tsx   |   82.60 |    82.35 |   83.33 |   82.60 | 54-57,72
 StatisticsFilter.tsx  |   88.88 |    83.33 |   85.71 |   88.88 | 28
```

테스트 커버리지 결과 출력(일부)



테스트 케이스 우선순위 결정 과정

트러블 슈팅 | 2. 프론트엔드 개발 환경 개선 프로젝트 회고 링크

문제 상황

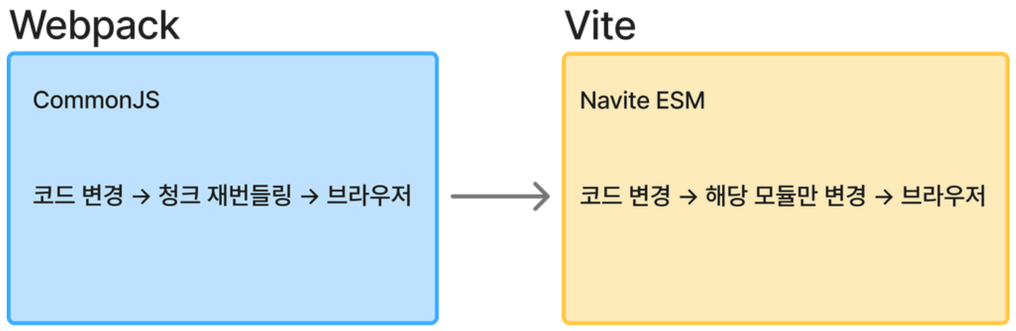
자사 프론트엔드 환경은 React + CRA 로 구성되어 있었습니다. 프로젝트 규모가 커짐에 따라 개발 서버 성능이 저하되었고, 이는 종종 일정 병목을 일으켰습니다. 개발 서버 시작은 평균 30초가 소요되었고, HMR은 평균 5~10초가 소요되었습니다. 이 문제를 해결하기 위해 우리 팀은 2개의 대안을 검토하게 되었습니다.

해결 과정

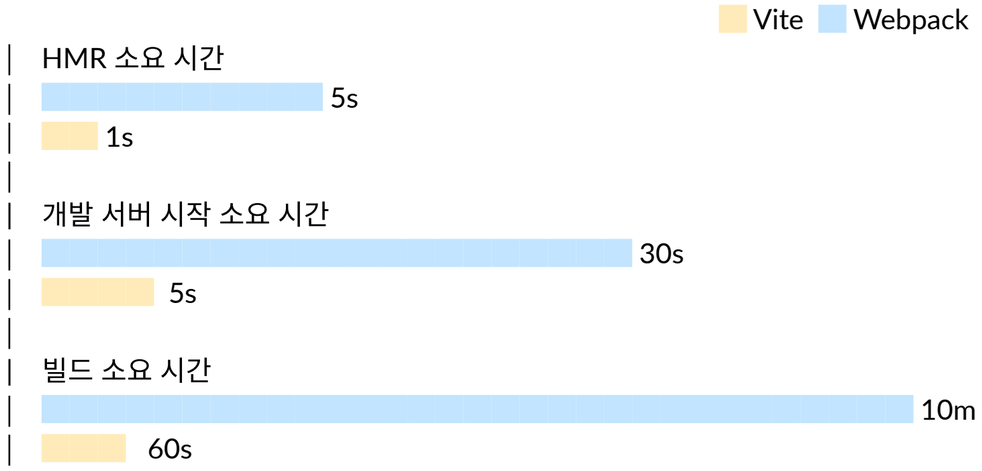
두 방법은 기존 ‘Webpack 최적화'와 ‘Vite 마이그레이션'이었습니다. 공식 문서와 관련 해외 포스팅을 통해 장단점을 다음과 같이 분석했습니다. Vite는 새로운 설정 파일 작성과 호환성 검토 등의 초기 비용이 크지만, ESM Native 구조를 통해 필요한 모듈만 동적으로 추가하여 프로젝트 규모에 관계 없이 비교적 빠른 개발 속도를 지원합니다.
Webpack은 초기 비용이 작지만, CommonJS 방식을 사용하여 모든 모듈을 항상 번들링하므로 프로젝트 규모에 따라 개발 서버 성능이 저하됩니다. 실제 테스트 결과, Vite가 최적화된 Webpack에 비해 평균 30% 이상 개발 서버 속도가 빨랐습니다. 또한 레거시가 된 Webpack을 사용했을 때 기술 부채 발생과 신규 팀원의 적응 시간도 고려해야 했습니다. 우리 팀은 ‘개발 서버 속도‘와 ‘추후 관리 비용‘을 판단 근거로 하여 Vite 마이그레이션을 결정했습니다. 또한 빌드 스크립트를 Vite의 rollup 기반 설정 파일로 재작성하고, 빌드 속도 개선을 위해 코드 스플리팅 및 모듈에 따른 청크 분할도 추가했습니다.

결과 및 배운 점

결과적으로 기존 Webpack 환경에 비해 Vite에서 오른쪽 차트와 같이 개선되었고, 코드 스플리팅을 통해 초기 로딩 시간 또한 개선되었습니다. 각 번들러 환경과 장단점을 이해할 수 있었고, 트레이드 오프 분석을 통한 기술 결정에 대해서도 배울 수 있었습니다.



Webpack, Vite 동작 방식



Webpack → Vite 성능 변화

트러블 슈팅 | 3. XSS 보안 취약점 개선 🔗트러블 슈팅 회고 링크 🔗링크 2

문제 상황

html 마크업 데이터를 다루는 텍스트 에디터 기능이 추가됨에 따라, XSS 공격에 대비하기 위해 보안 점검을 진행했습니다. 그 결과, 웹 에디터에 <script>를 삽입하여 악성 코드를 서버에 저장할 수 있는 공격법이 식별되었고, 이것을 막기 위해 웹 에디터 내장 옵션을 통해 html 태그가 자동으로 인코딩되어 저장되도록 설정했습니다. 하지만 이것만으로는 XSS 공격을 완벽히 차단할 수 없었습니다.

해결 과정

Burp Suite라는 패킷 스니핑 툴을 사용하여 클라이언트의 http 요청을 가로채 원하는 값(html 태그)을 강제로 서버에 삽입했습니다. 이때는 인코딩되지 않고 서버에 그대로 저장되어 렌더링 될 때 해당 스크립트가 실행됐습니다. 따라서, 요청 시점뿐만 아니라 렌더링 시점에 sanitize를 적용하여 스크립트 부분이 제거되도록 했습니다. 하지만 서버에서 클라이언트의 동작을 100% 신뢰할 수 없고, 추후 개발자의 실수 및 Log Injection 등과 같은 문제의 발생 가능성을 고려하여 서버에 html 마크업이 저장되지 않도록 수정했습니다. 화이트 리스트 방식을 통해 에디터에서 사용하는 미 위험 태그 및 속성을 제외하고 모두 차단하여, 총 3중 방식의 방어 코드를 적용했습니다.

결과 및 배운 점

위와 같은 과정을 통해 프론트엔드 단과 백엔드 단에서 XSS 취약점을 개선할 수 있었습니다. 프론트엔드 단에서 방어하더라도, 다른 우회 방식이나 개발자의 실수로 인해 보안 문제가 재발할 수 있습니다. 보안은 실수가 없어야 하는 것보다 실수가 있더라도 지켜져야 하는 구조여야 하고, 백엔드와 프론트엔드는 서로의 동작을 100% 신뢰할 수 없으므로 각자 방어 구조를 사용해야 한다는 것을 깨달았습니다.

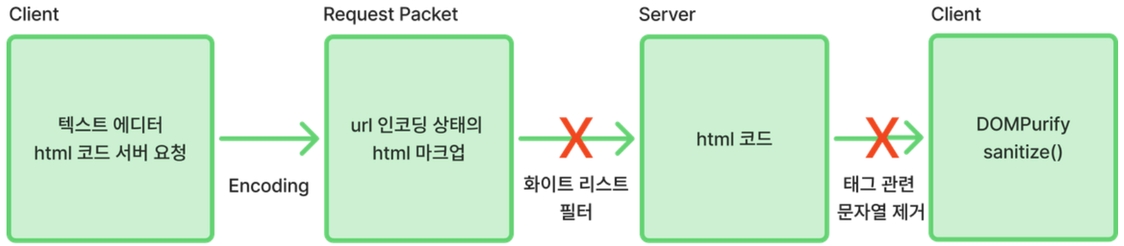
Request

Pretty Raw Hex

```
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: cors
17 Sec-Fetch-Dest: empty
18 Referer: [redacted]e/notice_create_view/
19 Accept-Encoding: gzip, deflate, br
20 Priority: u=1, i
21 Connection: keep-alive
22
23 ------WebKitFormBoundaryZ7SSV20h5r08vdM5
24 Content-Disposition: form-data; name="csrfm[redacted]n"
25
26 I12f2uwGQz8mWzFb6XKqVpNYRgOpCE2n
27 ------WebKitFormBoundaryZ7SSV20h5r08vdM5
28 Content-Disposition: form-data; name="document_html"
29
30 <p>&lt;script&gt;alert('asdfaf')&lt;/script&gt;</p>
31 ------WebKitFormBoundaryZ7SSV20h5r08vdM5
32 Content-Disposition: form-data; name="data_type"
```

Burp Suite 패킷 강제 변환 시뮬레이션

해당 부분 강제 변환



html 마크업 데이터로 인한 XSS 공격 루트 차단 과정

2024. 8 ~ 2024. 10

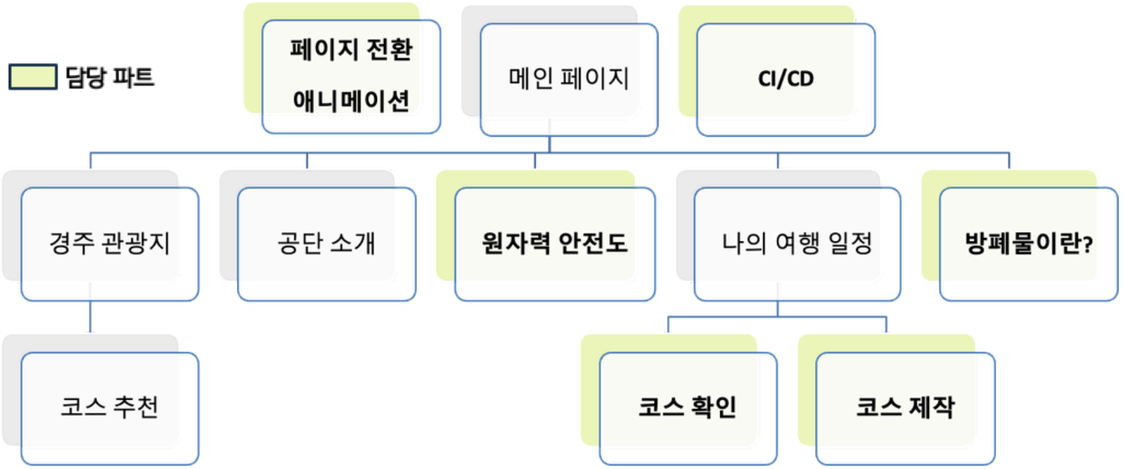
PROJECT 02.

KORAD - 경화주월

프로젝트 개요

잘 알려지지 않은 **원자력환경공단**을 경주 관광지로서 **홍보**하고, 공단의 **관리 안정성** 또한 나타내기 위해 제작한 웹앱 프로젝트

React, Javascript, React-Query, Docker, Nginx, Github-Actions, AWS EC2, Artillery



트러블 슈팅 | 1. 방사선 페이지 리렌더링 성능 개선 🔗트러블 슈팅 회고 🔗링크 2

문제 상황

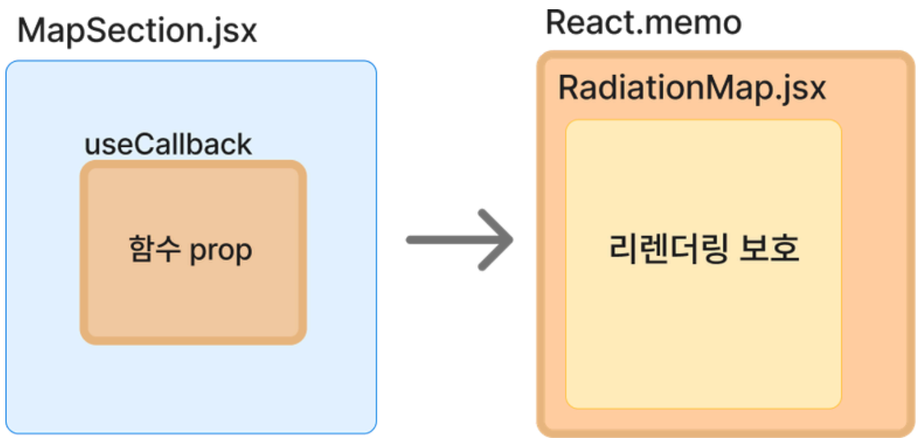
방사선량 시각화 페이지의 렌더링 시간이 다소 지연되던 이슈가 있었습니다. 원인을 찾기 위해 React Devtools를 사용했고, RadiationMap 컴포넌트가 총 5회 렌더링 되는 것을 발견할 수 있었습니다. 이는 예상했던 것보다 많은 렌더링 횟수였으며, 렌더링 원인은 props 및 state 변경이었습니다.

해결 과정

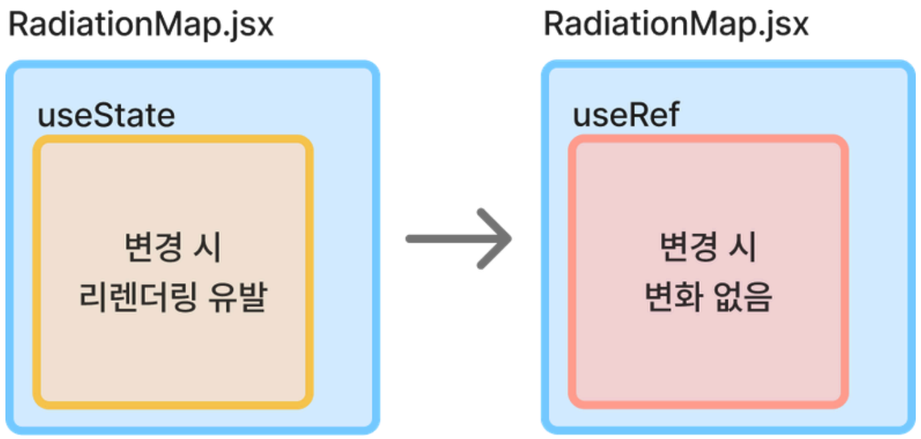
먼저 해당 컴포넌트는 함수 타입의 props를 받았는데, 리액트에서는 얕은 참조로 props를 가져오기 때문에 매 렌더링마다 다른 props로 인식되어 리렌더링이 발생했습니다. 이를 해결하기 위해 React.memo와 useCallback을 사용해 부모 컴포넌트의 리렌더링에도 함수 참조값이 변경되지 않도록 했고, 이에 따라 자식 컴포넌트는 변경된 props로 인식하지 않았습니다. 둘째로, 컴포넌트의 state 남용으로 발생한 리렌더링이었습니다. 본래 state는 리렌더링을 유발해야 하는 상황에서만 사용해야 하지만, 이전 코드에서는 값을 저장하여 useEffect 실행을 유발하는 용도로 사용했습니다. 이는 어떤 상태가 변경되어 함수가 실행되는지 알 수 있어 가독성은 증가하겠지만 불필요한 렌더링을 유발하여 웹 성능을 저하시키고, 렌더링 비용이 큰 해당 컴포넌트에서는 나쁜 선택지였습니다. 값은 저장하지만 렌더링을 유발하지 않는 useRef로 변경하고 두 개의 useEffect를 통합하는 방식으로 수정하여 리렌더링 횟수를 감소시켰습니다.

결과 및 배운 점

최종적으로 렌더링 시간은 60% 개선, 렌더링 횟수는 3회 감소시킬 수 있었습니다. 카카오맵이 포함되어 리렌더링 비용이 크기 때문에 불필요한 리렌더링을 최소화해야 했습니다. 해당 이슈를 통해, 렌더링 성능을 위해 기술적인 최적화뿐만 아니라 올바른 상태(state, props 등)를 사용하고 컴포넌트를 설계하는 것도 중요하다는 것을 깨닫게 되었습니다.



useCallback, React.memo 적용



useState → useRef 변경

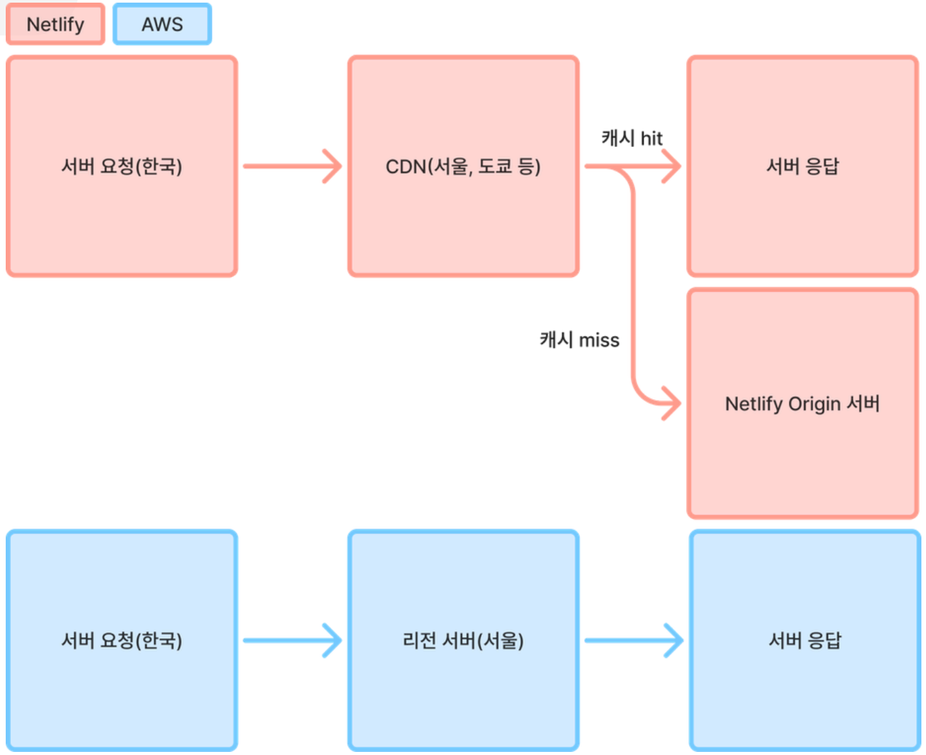
트러블 슈팅 | 2. 이미지 로딩 성능 이슈 해결 트러블 슈팅 회고

문제 상황

프론트엔드 netlify 배포 이후, 이미지 로딩 성능 저하 이슈를 발견했습니다. lighthouse를 통해 Core Web Vitals를 확인해 보니, CLS, TTFB, LCP가 낮은 성능으로 나타났으며 전체 점수는 평균 60점 미만이었습니다. 이 지표를 토대로 다음과 같은 단계를 거쳐 개선했습니다.

해결 과정

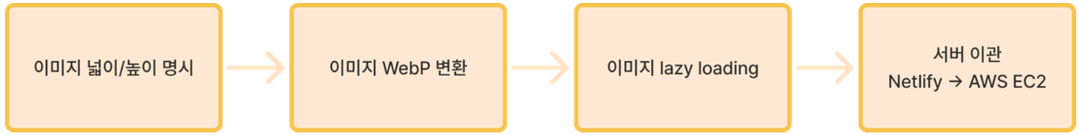
가장 먼저 Layout Shift 방지를 위해 이미지의 높이와 넓이를 명시하여 CLS를 개선했습니다. 또한, 이미지의 형식을 webp로 변환하여 손실/무손실 압축을 적용했고, 이미지 lazy loading을 사용했습니다. 그 결과 lighthouse 점수는 상승했지만 여전히 로딩 성능 이슈는 해결되지 않았습니다. 이에 따라 로딩 성능 저하의 원인은 서버 성능 저하라고 판단하게 되었습니다. Netlify는 CDN을 사용하여 해외 이용자에게도 비교적 빠른 응답을 지원하지만 네트워크 상황에 따라 대부분 도쿄에서 캐싱된 응답을 반환하고, 캐시 미스 시에는 해외 메인 서버에 요청됩니다. 서버 요청에 있어 거리는 곧 성능과 직결되기 때문에 리전 서버를 서울에 두고 있는 AWS보다는 평균 응답 속도가 느릴 것이라 판단했고, 이러한 근거를 통해 AWS S3로 이전하여 서버 성능 및 로딩 성능을 개선할 수 있었습니다.



AWS가 Netlify보다 한국에서 더 빠른 이유

결과 및 배운 점

결과적으로 lighthouse 성능 점수를 평균 60점에서 90점으로 개선할 수 있었고, 평균 LCP(3.2s → 1.5s), TTFB(950ms → 250ms), CLS(0.3 → 0.08)가 개선되었습니다. 또한 로딩 성능이 단지 기술적 이슈만이 아니라는 것을 깨닫게 되었으며 로딩 성능을 개선하면서 Core Web Vitals에 대해 깊이 이해할 수 있었습니다.



이미지 로딩 성능 이슈 해결 프로세스

2024. 10 ~ 2026. 2

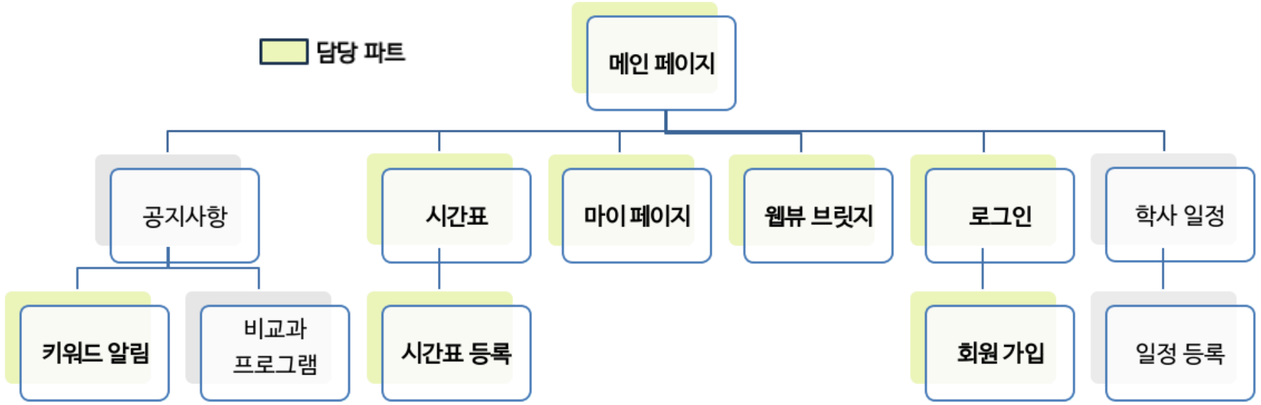
PROJECT 03.

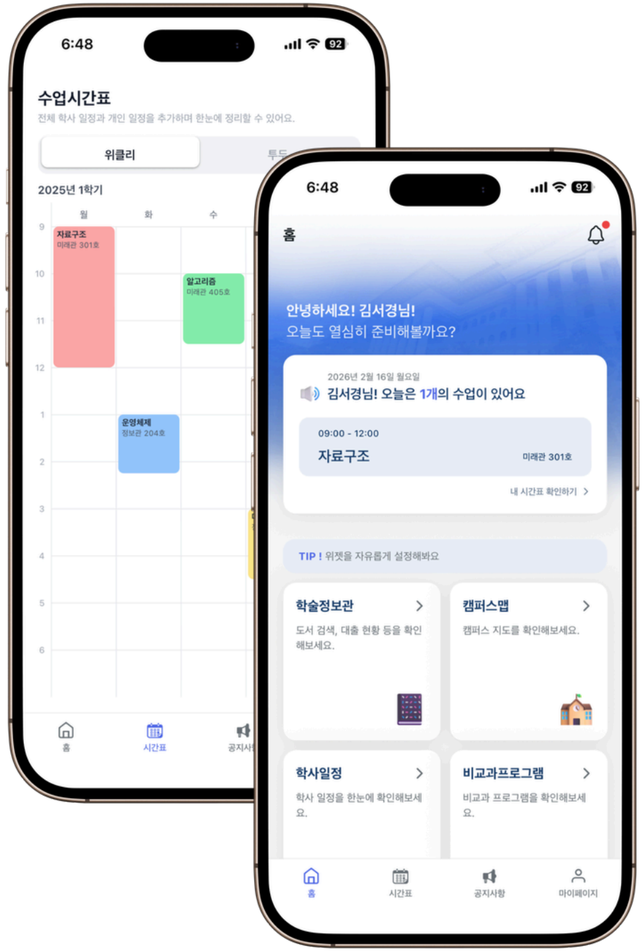
SKUMARU - 학교 사이트 재구축

프로젝트 개요

대학교 생활에 있어서 흩어지고 복잡한 **정보**들을 **하나**로 모으고, 학우들의 불편함을 개선하기 위해 제작한 프로젝트

Next, Typescript, React-Native, React-Query, MSW, Zustand, Expo, Jest





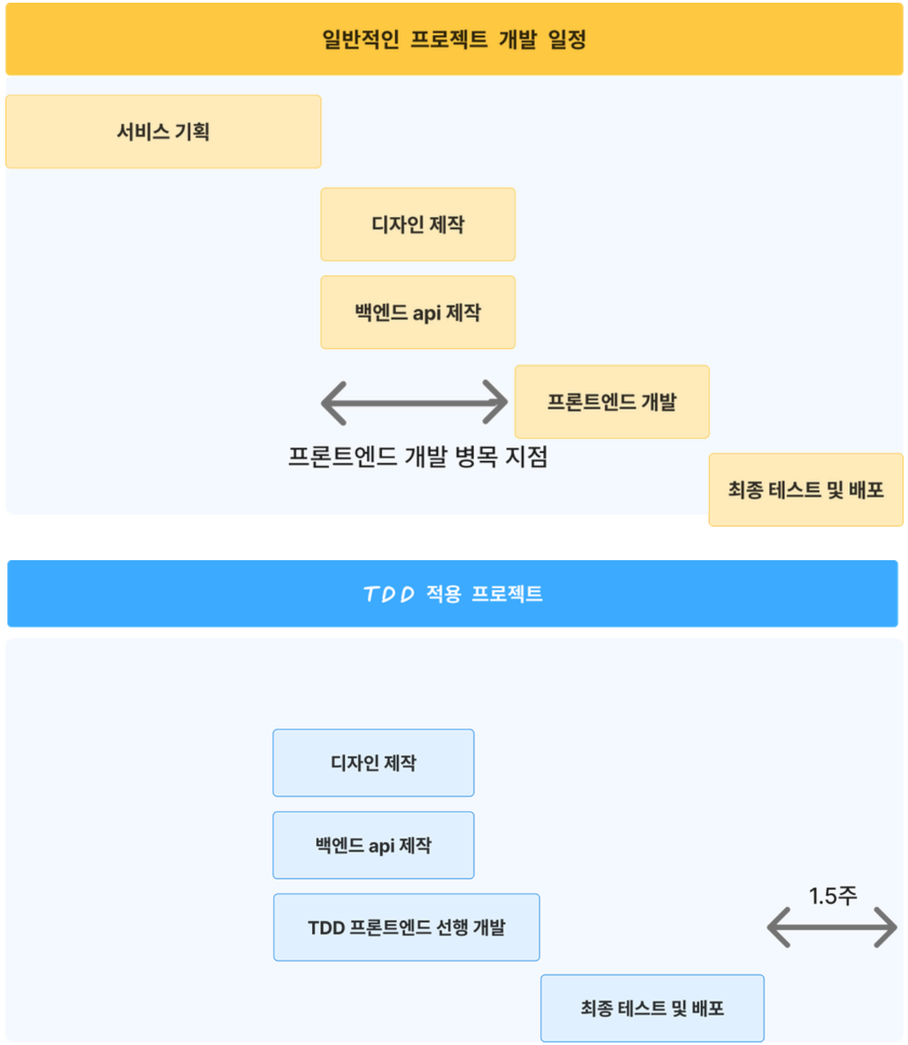
트러블 슈팅 | MSW 및 TDD로 개발 병목 완화

문제 상황

프로젝트 개발 일정이 8주 남은 상황에서, 디자인과 백엔드 api가 완성되지 않았습니다. 프론트엔드 팀은 개발 병목을 최대한 줄이기 위해 기술 스택 및 코드 컨벤션을 정립하고, 기본적인 세팅을 마쳤음에도 여전히 기다려야 했습니다.
이러한 지연 문제를 해결하기 위해 CDD와 TDD를 검토하게 되었습니다.

해결 과정

TDD는 api 제작 병목 시에 주요 비즈니스 로직을 선행 개발하며 테스트 코드를 먼저 작성할 수 있습니다. 때문에 추후 버그 식별이 용이하여 QA 기간을 단축할 수 있지만, 초기 비용이 크고 높은 학습 곡선이 요구됩니다. CDD는 디자인 시안만 나오면 바로 컴포넌트 단위로 제작과 검증을 독립적으로 할 수 있고 디자이너와의 실시간 피드백이 가능하지만, 비즈니스 로직이 없기 때문에 이후에 수정 비용이 발생할 수 있고 테스트 비용도 무시할 수 없었습니다. 우리 팀은 장단점을 분석하여 TDD를 개발하기로 했고, 판단 기준은 '얼마나 병목을 단축할 수 있는가'였습니다. 백엔드 api를 대부분의 페이지에서 사용하므로 많은 모킹 데이터가 필요했고, 이 데이터를 통해 제작할 수 있는 비즈니스 로직이 많고 분명했기 때문에 선행 개발의 효과를 극대화할 수 있을 것이라 판단했습니다. 또한 CDD에서는 추후 UI 및 비즈니스 로직 수정 상황의 비용이 예측되지 않아 TDD를 더 안전한 방법이라고 판단했습니다. MSW를 도입하고 service, util 레이어를 추출하여 총 96개의 단위 테스트를 작성했고, 65% 이상의 코드 커버리지를 확보했습니다.

결과 및 배운 점

결과적으로 TDD를 활용한 선행 개발을 통해 프론트엔드 개발 일정을 1.5주 단축했습니다. 병목과 같은 상황에서도 마냥 기다리는 것이 아니라, 팀을 위해 할 수 있는 것들을 최대한 해야 한다는 것과 테스트를 통해 개발 안정성을 확보하는 방법을 배울 수 있었습니다.